CSA – 0610

DESIGN AND ANALYSIS OF ALGORITHMS

ASSIGNMENT – 4 B

DONE BY: DIVYA DHARSHINI V S

ROLL NO: 192511308

DESIGN AND ANALYSIS OF ALGORITHMS

ASSIGNMENT 4 A

ALGORITHM ENGINEERING: IMPLEMENT, TEST, DEBUG & OPTIMISE

TASK 1 – BUILD AND VALIDATE A DYNAMIC PROGRAMMING SOLUTION

Word Wrap as a Document-Layout Engine

1.1 Implement the Solution

1. Introduction

Word Wrap is a text-layout optimisation problem that arranges words into lines without exceeding the maximum width W. Dynamic Programming is used to minimise the total unused-space penalty.

For a non-final line:

Penalty = (Unused Spaces)²

The final line has zero penalty.

2. Objective

To arrange all words into lines such that the total penalty is minimum.

3. Mathematical Formulation

Let:

- n = Number of words.
- W = Maximum line width.
- L(i,j) = Total length of words i to j, including spaces.
- DP[j] = Minimum penalty for arranging the first j words.

If words i to j fit within the width:

DP[j] = min { DP[i-1] + Cost(i,j) }

Where:

Cost(i,j) = $(W - L(i,j))^2$, if j is not the final word.

Cost(i,j) = 0, if j = n.

Base condition:

DP[0] = 0.

4. Algorithm

1. START.
2. Read the words and maximum width W.
3. Initialise DP[0] = 0 and other states to infinity.
4. Calculate the length of possible word groups.
5. Ignore groups exceeding W.
6. Calculate the penalty for valid groups.
7. Update the minimum DP value and predecessor.
8. Reconstruct the optimal lines.
9. Display each line, unused spaces and penalty.
10. STOP.

5. Pseudocode

```
BEGIN
 DP[0] = 0
 FOR j = 1 TO n
   DP[j] = INF
   FOR i = j DOWNTO 1
     IF L(i,j) <= W
       cost = 0 if j=n else (W-L(i,j))²
```

IF DP[i-1]+cost < DP[j]

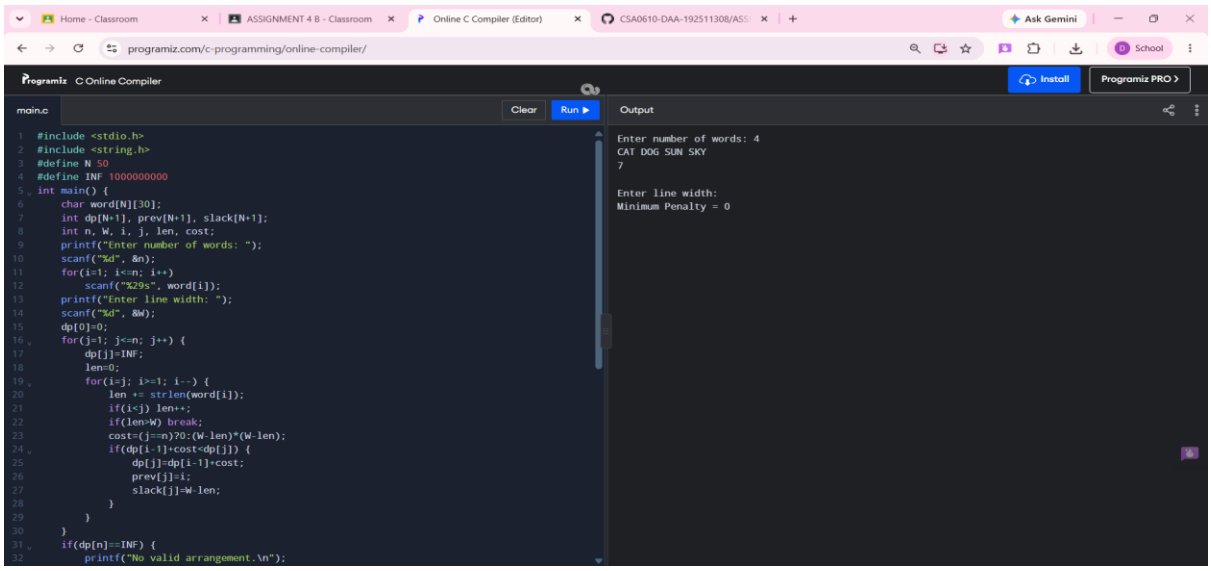
DP[j] = DP[i-1]+cost

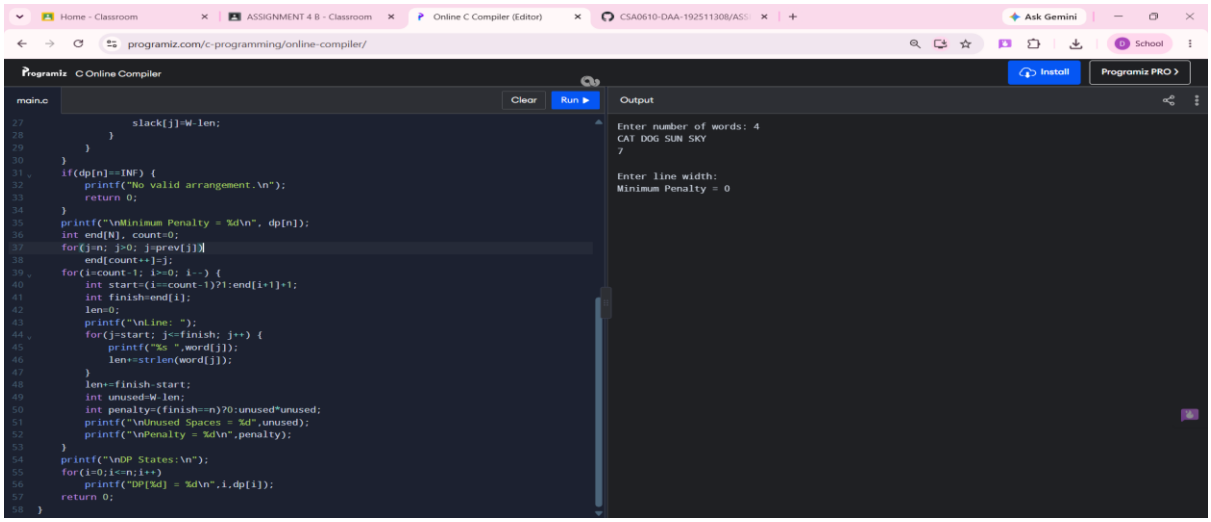
prev[j] = i

OUTPUT DP[n] and reconstructed lines

END

6. Complete C Program





7. Sample Input

4

CAT DOG SUN SKY

7

8. Sample Output

Minimum Penalty = 0

Line: CAT DOG

Unused Spaces = 0

Penalty = 0

Line: SUN SKY

Unused Spaces = 0

Penalty = 0

DP States:

DP[0] = 0

DP[1] = 16

DP[2] = 0

DP[3] = 16

DP[4] = 0

9. Output Explanation

The first two words occupy exactly seven characters, including the space. The remaining two words also occupy seven characters. Both lines have zero unused spaces. Therefore, the minimum total penalty is zero.

1.2 Expose the DP Behaviour

The DP array stores the minimum penalty required to arrange the first j words.

DP State	Meaning	Value
DP[0]	No words arranged	0
DP[1]	CAT	16
DP[2]	CAT DOG	0
DP[3]	CAT DOG SUN	16
DP[4]	CAT DOG SUN SKY	0

Interpretation:

- DP[1] creates a line containing CAT with 4 unused spaces.
- DP[2] groups CAT and DOG, reducing the penalty to zero.
- DP[3] considers alternative arrangements for three words.
- DP[4] finds the final optimal arrangement.

The predecessor array stores where each optimal line begins. It allows the program to reconstruct the actual arrangement rather than displaying only the cost.

Why Dynamic Programming?

A greedy method may select the longest possible line without considering the penalties of subsequent lines. DP evaluates multiple valid arrangements and retains the minimum cost.

1.3 Requirement Change

Prediction Before Execution

The same text is tested with three different widths.

Width	Predicted Arrangement	Expected Penalty
6	Four separate lines	27
7	Two pairs of words	0
9	Two pairs of words	4

Modified Case 1: W = 6

Expected arrangement:

CAT

DOG

SUN

SKY

Minimum Penalty = 27

Each of the first three lines has three unused spaces.

Penalty = $3^2 + 3^2 + 3^2$

= 27.

Modified Case 2: W = 7

CAT DOG

SUN SKY

Minimum Penalty = 0

Both lines exactly occupy the available width.

Modified Case 3: W = 9

CAT DOG

SUN SKY

Minimum Penalty = 4

The first line has two unused spaces.

Penalty = $2^2 = 4$.

Interpretation

The observed results agree with the predictions.

Increasing the width does not necessarily reduce the penalty monotonically because the final line carries no penalty and the grouping decisions change.

1.4 Boundary Test

Selected Boundary Case:

Number of Words = 1

Word = ALGORITHM

Width = 9

Expected output:

Minimum Penalty = 0

Line: ALGORITHM

Unused Spaces = 0

Penalty = 0

Reason:

A single word becomes the final line and must receive zero penalty. This test checks whether the implementation correctly handles the final-line exception and the base DP state.

Time Complexity Proof

There are n DP states.

For every state j, the algorithm examines at most j previous positions.

$T(n) = 1 + 2 + 3 + ... + n$

$T(n) = n(n+1)/2$

$T(n) = (n^2+n)/2$

Therefore:

Time Complexity = $O(n^2)$

Space Complexity

The word array, DP array and predecessor array require O(n) memory.

Therefore:

Space Complexity = O(n)

Conclusion

Dynamic Programming produces an optimal document arrangement by evaluating different word groupings. The intermediate states and reconstructed lines demonstrate how the minimum penalty is obtained.

TASK 2 – GREEDY VS DYNAMIC PROGRAMMING

Build It, Break It, Explain It

2.1 Normal Case

1. Introduction

Knapsack is an optimisation problem where items must be selected within a limited capacity to maximise total value.

Two approaches are implemented:

- Fractional Knapsack using Greedy.
- 0/1 Knapsack using Dynamic Programming.

2. Dataset

Capacity = 10 units.

Item	Weight	Value	Value/Weight
A	2	40	20
B	3	45	15
C	5	50	10

3. Greedy Decision Sequence

Items are arranged in decreasing value-to-weight ratio.

1. Select A: Weight = 2, Value = 40.
2. Select B: Weight = 3, Value = 45.
3. Select C: Weight = 5, Value = 50.

Total weight = 2 + 3 + 5 = 10.

Total value = 40 + 45 + 50 = 135.

Both approaches produce an optimal value of 135 for this dataset.

2.2 Break the Greedy Intuition

Counterexample Dataset

Capacity = 50.

Item	Weight	Value	Ratio
A	10	60	6
B	20	100	5
C	30	120	4

Greedy 0/1 Selection

1. Select A because it has the highest ratio.
2. Select B because it is the next highest ratio.
3. C cannot fit in the remaining capacity.

Greedy result:

Weight = 10 + 20 = 30.

Value = 60 + 100 = 160.

DP Optimal Selection

Select B and C.

Weight = 20 + 30 = 50.

Value = 100 + 120 = 220.

Therefore:

GreedyResult = 160.

DPOptimum = 220.

GreedyResult ≠ DPOptimum

The first incorrect commitment occurs when Greedy selects A. Although A has the highest individual ratio, it prevents the combination of B and C that produces a higher total value.

2.3 Engineering Comparison

Factor	Fractional Greedy	0/1 Greedy	0/1 DP
Selection	Fractions allowed	Whole items	Whole items
Main operation	Ratio comparison	Ratio comparison	State transition
Memory	O(1) auxiliary	O(1) auxiliary	O(nW)
Optimality	Guaranteed	Not guaranteed	Guaranteed
Complexity	O(n log n)	O(n log n)	O(nW)

For fractional knapsack, sorting dominates the work.

For DP, the number of states is approximately nW.

3. Why the Results Differ

Fractional Knapsack allows part of an item to be selected. Therefore, selecting the highest value-to-weight ratio first is optimal.

0/1 Knapsack does not allow fractions. A locally attractive selection may prevent a better combination.

2.4 Changed Requirement

Management requires an extremely fast near-optimal answer.

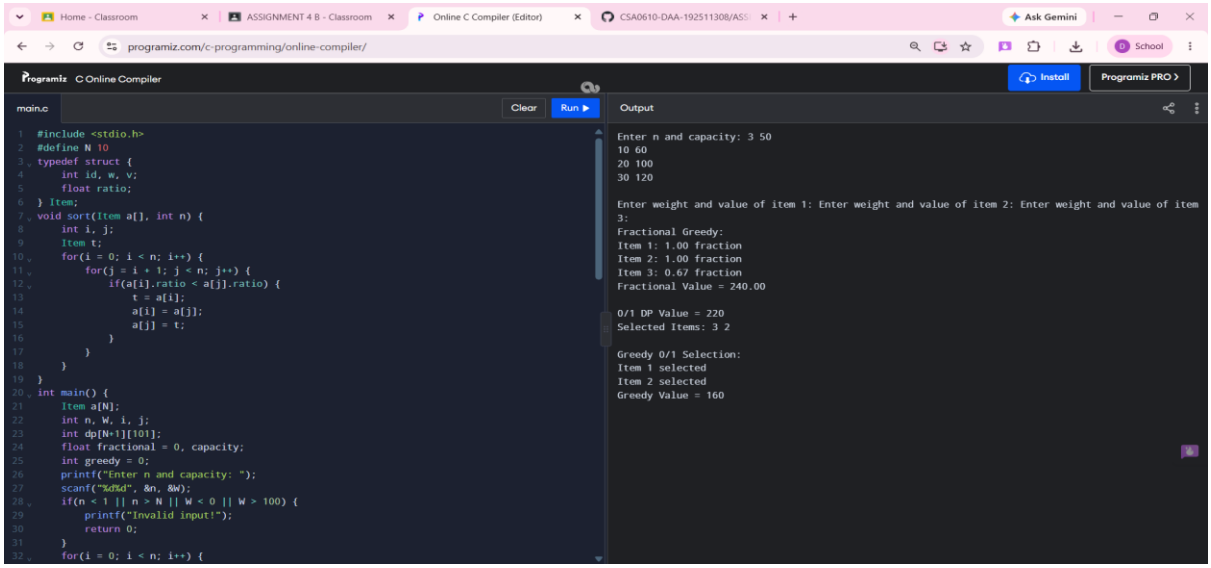
The ratio-based greedy method is suitable when speed is more important than guaranteed optimality.

For the counterexample, it returns 160 instead of 220.

The DP approach guarantees 220 but performs more computational work and requires additional memory.

Therefore, I would investigate the greedy method for rapid approximate decisions, while retaining DP when exact optimality is essential.

Complete C Program



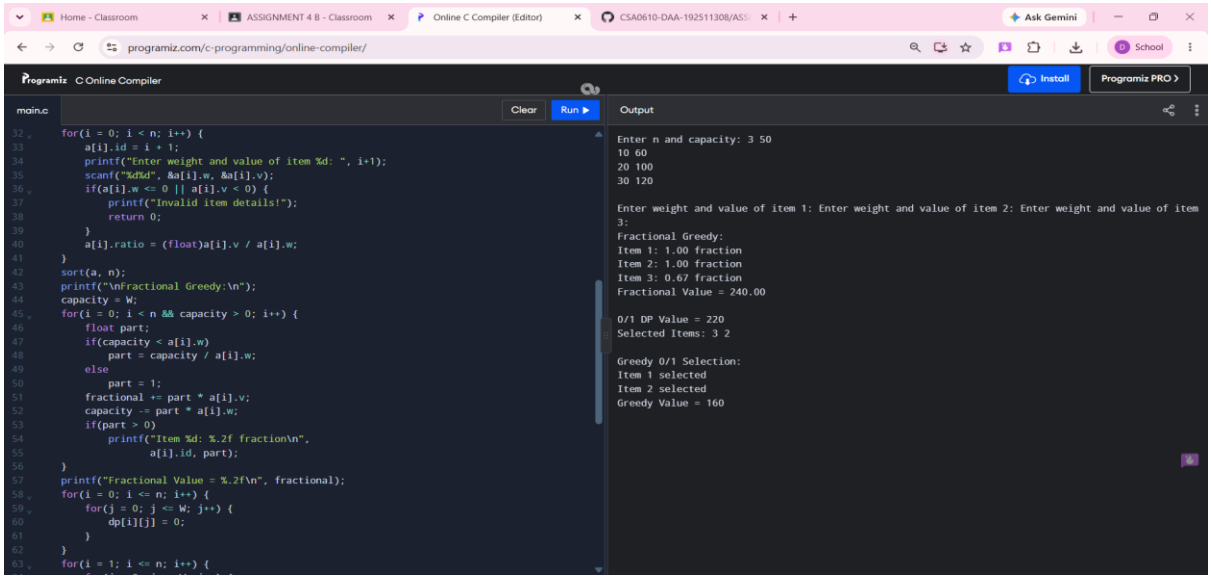
```c
#include <stdio.h>
#define N 10
typedef struct {
    int id, w, v;
    float ratio;
} Item;
void sort(Item a[], int n) {
    int i, j;
    Item t;
    for(i = 0; i < n; i++) {
        for(j = i + 1; j < n; j++) {
            if(a[i].ratio < a[j].ratio) {
                t = a[i];
                a[i] = a[j];
                a[j] = t;
            }
        }
    }
}
int main() {
    Item a[N];
    int n, W, i, j;
    int dp[N+1][101];
    float fractional = 0, capacity;
    int greedy = 0;
    printf("Enter n and capacity: ");
    scanf("%d%d", &n, &W);
    if(n < 1 || n > N || W < 0 || W > 100) {
        printf("Invalid input!");
        return 0;
    }
    for(i = 0; i < n; i++) {
```

```
Enter n and capacity: 3 50
10 60
20 100
30 120

Enter weight and value of item 1: Enter weight and value of item 2: Enter weight and value of item
3:
Fractional Greedy:
Item 1: 1.00 fraction
Item 2: 1.00 fraction
Item 3: 0.67 fraction
Fractional Value = 240.00

0/1 DP Value = 220
Selected Items: 3 2

Greedy 0/1 Selection:
Item 1 selected
Item 2 selected
Greedy Value = 160
```



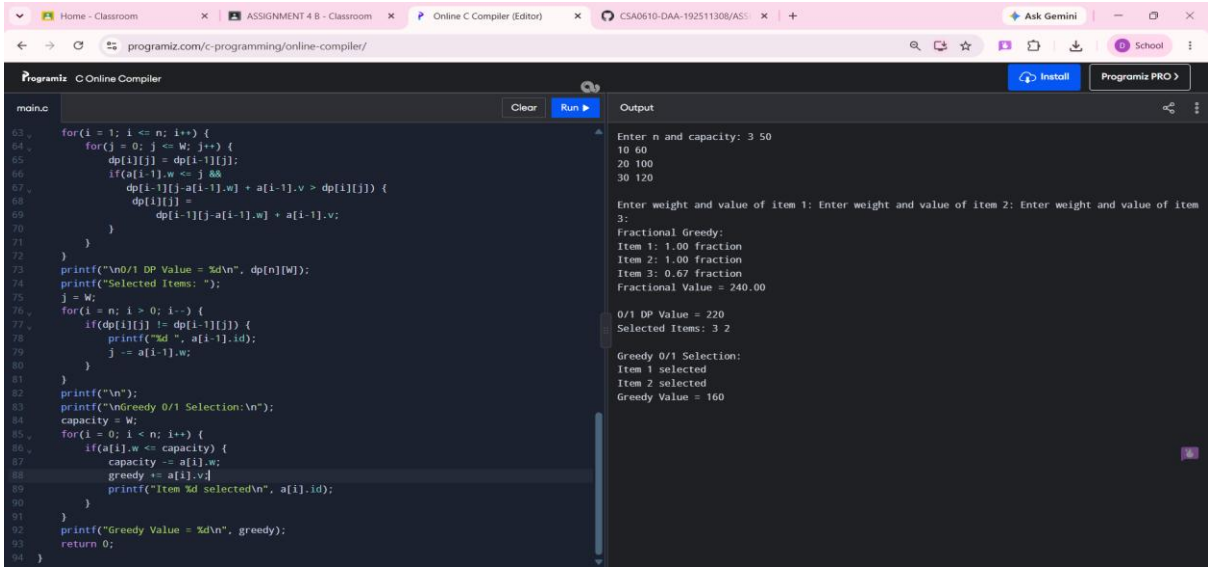
```c
    for(i = 0; i < n; i++) {
        a[i].id = i + 1;
        printf("Enter weight and value of item %d: ", i+1);
        scanf("%d%d", &a[i].w, &a[i].v);
        if(a[i].w <= 0 || a[i].v < 0) {
            printf("Invalid item details!");
            return 0;
        }
        a[i].ratio = (float)a[i].v / a[i].w;
    }
    sort(a, n);
    printf("\nFractional Greedy:\n");
    capacity = W;
    for(i = 0; i < n && capacity > 0; i++) {
        float part;
        if(capacity < a[i].w)
            part = capacity / a[i].w;
        else
            part = 1;
        fractional += part * a[i].v;
        capacity -= part * a[i].w;
        if(part > 0)
            printf("Item %d: %.2f fraction\n",
                   a[i].id, part);
    }
    printf("Fractional Value = %.2f\n", fractional);
    for(i = 0; i <= n; i++) {
        for(j = 0; j <= W; j++) {
            dp[i][j] = 0;
        }
    }
    for(i = 1; i <= n; i++) {
```

```
Enter n and capacity: 3 50
10 60
20 100
30 120

Enter weight and value of item 1: Enter weight and value of item 2: Enter weight and value of item
3:
Fractional Greedy:
Item 1: 1.00 fraction
Item 2: 1.00 fraction
Item 3: 0.67 fraction
Fractional Value = 240.00

0/1 DP Value = 220
Selected Items: 3 2

Greedy 0/1 Selection:
Item 1 selected
Item 2 selected
Greedy Value = 160
```



```c
    for(i = 1; i <= n; i++) {
        for(j = 0; j <= W; j++) {
            dp[i][j] = dp[i-1][j];
            if(a[i-1].w <= j &&
               dp[i-1][j-a[i-1].w] + a[i-1].v > dp[i][j]) {
                dp[i][j] =
                    dp[i-1][j-a[i-1].w] + a[i-1].v;
            }
        }
    }
    printf("\n0/1 DP Value = %d\n", dp[n][W]);
    printf("Selected Items: ");
    j = W;
    for(i = n; i > 0; i--) {
        if(dp[i][j] != dp[i-1][j]) {
            printf("%d ", a[i-1].id);
            j -= a[i-1].w;
        }
    }
    printf("\n");
    printf("\nGreedy 0/1 Selection:\n");
    capacity = W;
    for(i = 0; i < n; i++) {
        if(a[i].w <= capacity) {
            capacity -= a[i].w;
            greedy += a[i].v;
            printf("Item %d selected\n", a[i].id);
        }
    }
    printf("Greedy Value = %d\n", greedy);
    return 0;
}
```

```
Enter n and capacity: 3 50
10 60
20 100
30 120

Enter weight and value of item 1: Enter weight and value of item 2: Enter weight and value of item
3:
Fractional Greedy:
Item 1: 1.00 fraction
Item 2: 1.00 fraction
Item 3: 0.67 fraction
Fractional Value = 240.00

0/1 DP Value = 220
Selected Items: 3 2

Greedy 0/1 Selection:
Item 1 selected
Item 2 selected
Greedy Value = 160
```

Sample Input

3 50

10 60

20 100

30 120

Sample Output

Fractional Greedy:

Item 1: 1.00 fraction

Item 2: 1.00 fraction

Item 3: 0.67 fraction

Fractional Value = 240.00

0/1 DP Value = 220

Selected Items: 3 2

Greedy 0/1 Selection:

Item 1 selected

Item 2 selected

Greedy Value = 160

Output Explanation

Fractional Greedy takes all of A and B and two-thirds of C. The 0/1 DP selects B and C, achieving 220. The greedy whole-item approach selects A and B, achieving only 160.

Time Complexity Proof

Sorting:

$T_1(n) = O(n^2)$ for the simple sorting implementation.

DP:

There are (n+1)(W+1) states.

Each state requires constant-time comparison.

$T_2(n,W) = (n+1)(W+1) \times O(1)$.

Therefore:

DP Time Complexity = O(nW)

Overall implementation:

$O(n^2 + nW)$

Space Complexity

The DP table contains (n+1)(W+1) entries.

Therefore:

O(nW)

Conclusion

The experiment demonstrates that Greedy is optimal for Fractional Knapsack but does not guarantee an optimal solution for 0/1 Knapsack. Dynamic Programming evaluates combinations and guarantees the optimum.

TASK 3 – ROUTING ENGINE UNDER CHANGING NETWORK CONDITIONS

Dijkstra + Bellman-Ford + Floyd + Warshall

1. Introduction

A routing engine determines the minimum-cost path between locations. Different shortest-path algorithms are required depending on edge weights and the number of source-destination queries.

2. Graph Design

The network contains eight vertices and twelve directed edges.

Vertices:

A, B, C, D, E, F, G, H.

Original Edges

Edge	Weight
A → B	4
A → C	5
B → D	6
C → D	1
C → E	7
D → E	2
D → F	5
E → F	1
F → G	3
G → H	2
E → H	8
B → E	10

Stage A – Normal Routing

3. Dijkstra's Algorithm

Dijkstra finds shortest paths from a selected source when all edge weights are non-negative.

Source = A.

Algorithm

1. Initialise source distance to zero.
2. Set all other distances to infinity.
3. Select the unfinalised vertex with minimum distance.
4. Relax its outgoing edges.
5. Record predecessor information.
6. Repeat until all reachable vertices are finalised.
7. Reconstruct the shortest paths.

Pseudocode

```
BEGIN
 Set dist[A] = 0 and others = INF
 WHILE an unfinalised vertex exists
  Select minimum-distance vertex u
  Finalise u
  FOR each neighbour v of u
   IF dist[u]+weight(u,v) < dist[v]
    Update dist[v]
    parent[v] = u
 OUTPUT distances and paths
END
```

Expected Normal Results

Vertex	Distance	Shortest Path
A	0	A
B	4	A-B
C	5	A-C
D	6	A-C-D
E	8	A-C-D-E
F	9	A-C-D-E-F
G	12	A-C-D-E-F-G
H	14	A-C-D-E-F-G-H

Finalisation Order

A → B → C → D → E → F → G → H.

Stage B – Introduce a Negative Edge

Modify the edge: C → B = -3.

Prediction Before Execution

Dijkstra may become invalid because it can finalise B using the direct path before examining the negative alternative through C.

Original route:

A → B = 4.

Modified route:

A → C → B = 5 - 3 = 2.

Therefore, B's correct shortest distance becomes 2.

Bellman-Ford

Bellman-Ford repeatedly relaxes all edges for V-1 iterations.

It also detects reachable negative-weight cycles.

Expected Comparison

Algorithm	Distance to B
Dijkstra	4
Bellman-Ford	2

Dijkstra may return an incorrect result because its finalisation assumption is violated.

Stage C – Production Failure

Introduce:

H → C = -10.

The cycle is:

C → D → E → F → G → H → C.

Cycle weight:

= 1 + 2 + 1 + 3 + 2 - 10

= -1.

Since the cycle is reachable from A and has negative total weight, no finite shortest path exists for vertices affected by it.

Bellman-Ford detects this by performing one additional relaxation pass after V-1 iterations.

If any reachable distance can still decrease, a reachable negative-weight cycle exists.

Stage D – All-Pairs Routing

Floyd's Algorithm

Floyd's algorithm calculates shortest-path distances between every pair of vertices.

Recurrence:

$D(i,j) = \min(D(i,j), D(i,k) + D(k,j))$.

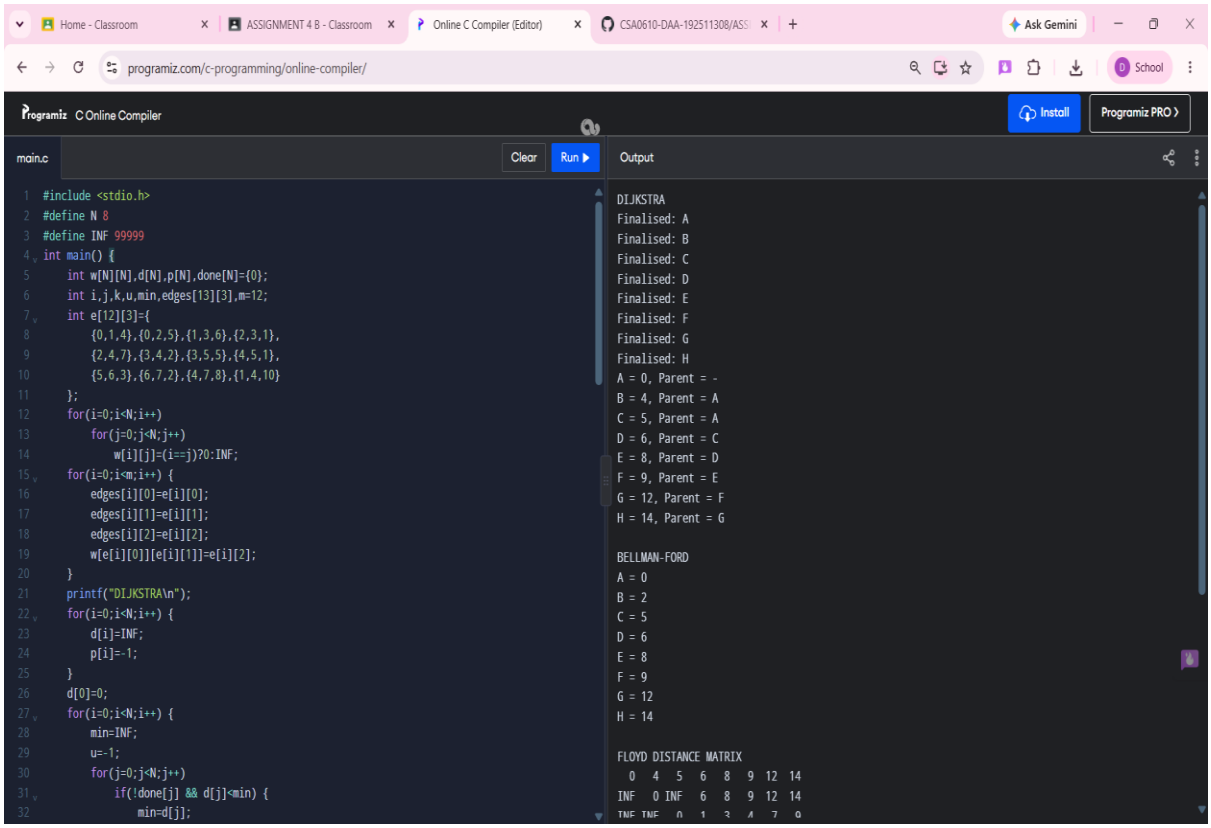
Warshall's Algorithm

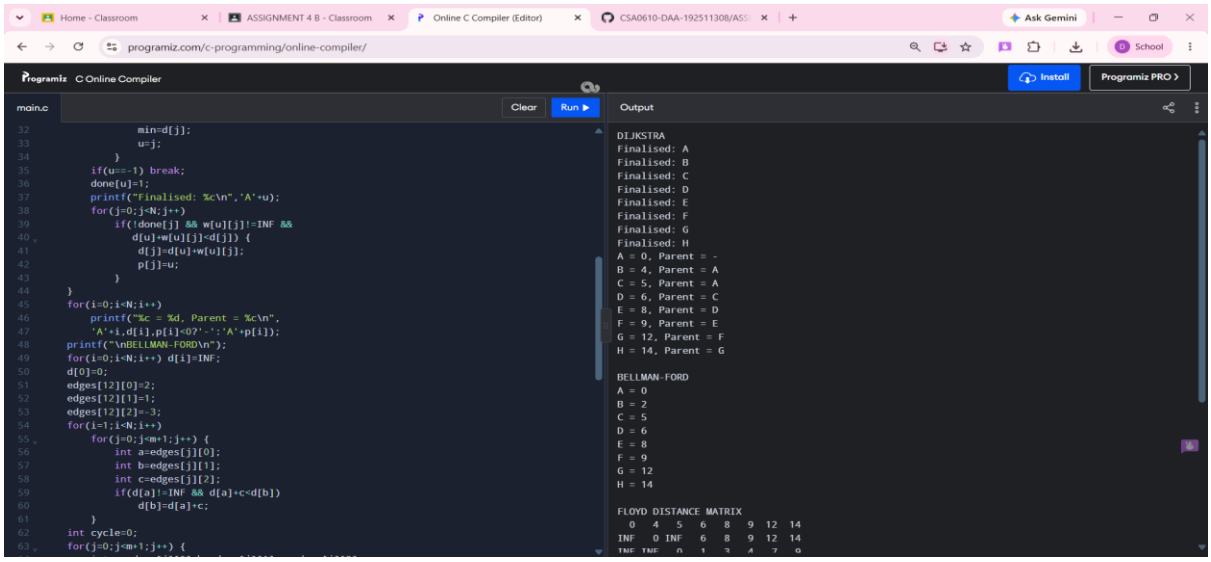
Warshall determines whether one vertex is reachable from another.

Recurrence:

$R(i,j) = R(i,j)$ OR $(R(i,k)$ AND $R(k,j))$.

Complete C Program



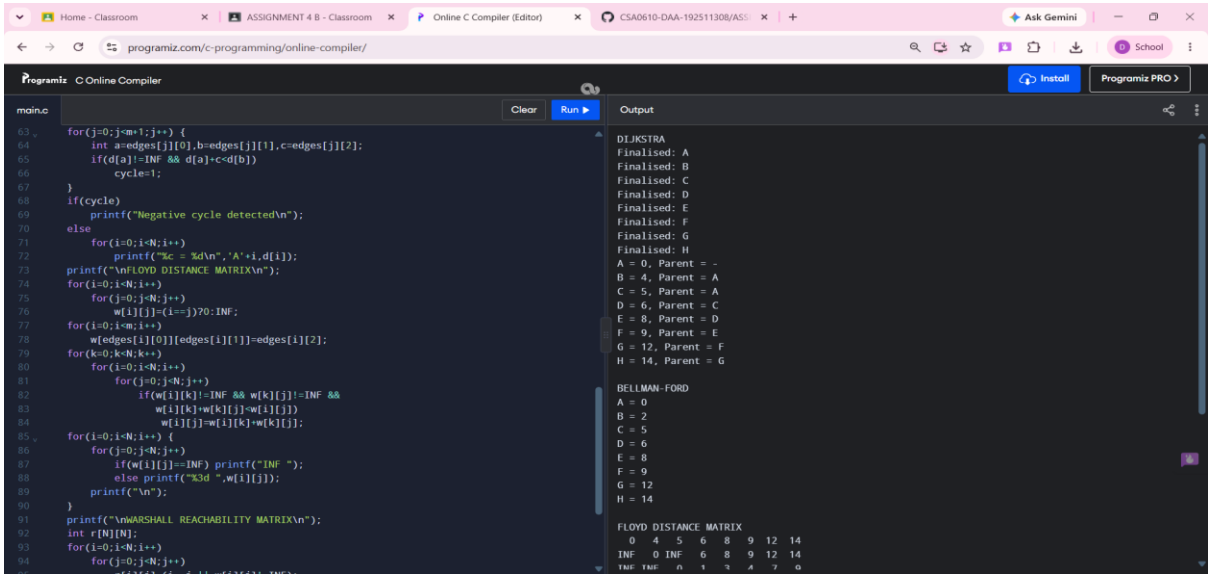


```
            min=d[j];
            u=j;
        }
    if(u==-1) break;
    done[u]=1;
    printf("Finalised: %c\n",'A'+u);
    for(j=0;j<N;j++)
        if(!done[j] && w[u][j]!=INF &&
           d[u]+w[u][j]<d[j]) {
            d[j]=d[u]+w[u][j];
            p[j]=u;
        }
}
for(i=0;i<N;i++)
    printf("%c = %d, Parent = %c\n",
    'A'+i,d[i],p[i]<0?'-':'A'+p[i]);
printf("\nBELLMAN-FORD\n");
for(i=0;i<N;i++) d[i]=INF;
d[0]=0;
edges[12][0]=2;
edges[12][1]=1;
edges[12][2]=-3;
for(i=1;i<N;i++)
    for(j=0;j<m+1;j++) {
        int a=edges[j][0];
        int b=edges[j][1];
        int c=edges[j][2];
        if(d[a]!=INF && d[a]+c<d[b])
            d[b]=d[a]+c;
    }
int cycle=0;
for(j=0;j<m+1;j++) {
```

```
DIJKSTRA
Finalised: A
Finalised: B
Finalised: C
Finalised: D
Finalised: E
Finalised: F
Finalised: G
Finalised: H
A = 0, Parent = -
B = 4, Parent = A
C = 5, Parent = A
D = 6, Parent = C
E = 8, Parent = D
F = 9, Parent = E
G = 12, Parent = F
H = 14, Parent = G

BELLMAN-FORD
A = 0
B = 2
C = 5
D = 6
E = 8
F = 9
G = 12
H = 14

FLOYD DISTANCE MATRIX
  0   4   5   6   8   9  12  14
INF   0 INF   6   8   9  12  14
```

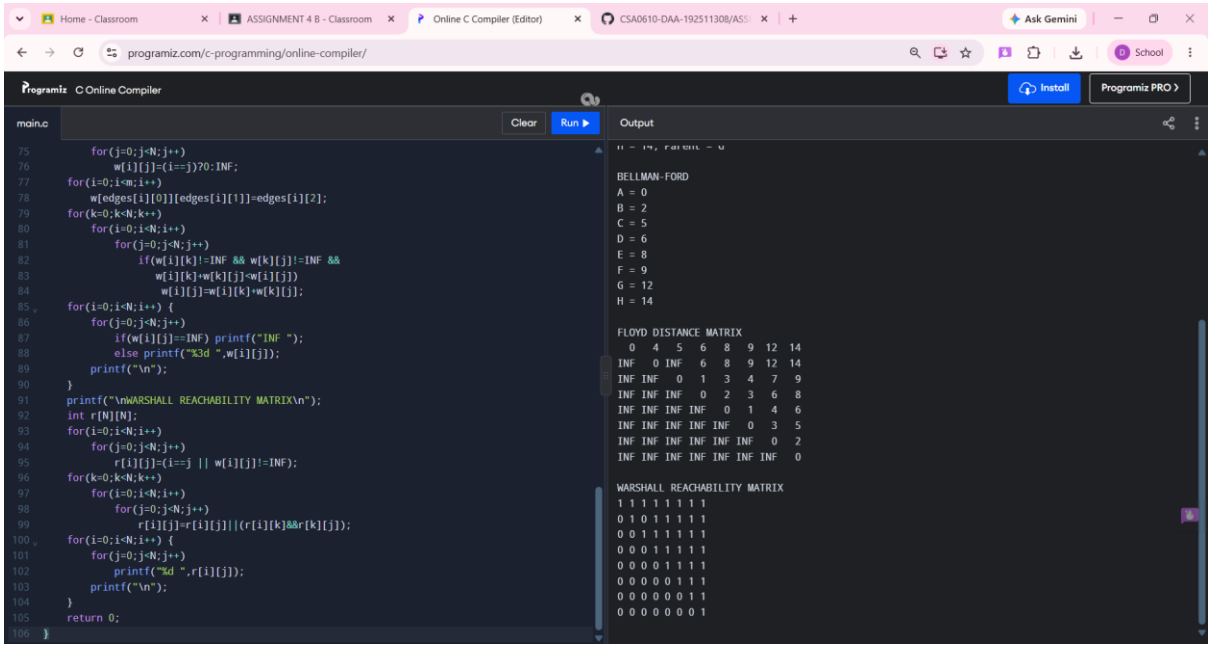


```
    for(j=0;j<m+1;j++) {
        int a=edges[j][0],b=edges[j][1],c=edges[j][2];
        if(d[a]!=INF && d[a]+c<d[b])
            cycle=1;
    }
    if(cycle)
        printf("Negative cycle detected\n");
    else
        for(i=0;i<N;i++)
            printf("%c = %d\n",'A'+i,d[i]);
    printf("\nFLOYD DISTANCE MATRIX\n");
    for(i=0;i<N;i++)
        for(j=0;j<N;j++)
            w[i][j]=(i==j)?0:INF;
    for(i=0;i<m;i++)
        w[edges[i][0]][edges[i][1]]=edges[i][2];
    for(k=0;k<N;k++)
        for(i=0;i<N;i++)
            for(j=0;j<N;j++)
                if(w[i][k]!=INF && w[k][j]!=INF &&
                   w[i][k]+w[k][j]<w[i][j])
                    w[i][j]=w[i][k]+w[k][j];
    for(i=0;i<N;i++) {
        for(j=0;j<N;j++)
            if(w[i][j]==INF) printf("INF ");
            else printf("%3d ",w[i][j]);
        printf("\n");
    }
    printf("\nWARSHALL REACHABILITY MATRIX\n");
    int r[N][N];
    for(i=0;i<N;i++)
        for(j=0;j<N;j++)
```

```
DIJKSTRA
Finalised: A
Finalised: B
Finalised: C
Finalised: D
Finalised: E
Finalised: F
Finalised: G
Finalised: H
A = 0, Parent = -
B = 4, Parent = A
C = 5, Parent = A
D = 6, Parent = C
E = 8, Parent = D
F = 9, Parent = E
G = 12, Parent = F
H = 14, Parent = G

BELLMAN-FORD
A = 0
B = 2
C = 5
D = 6
E = 8
F = 9
G = 12
H = 14

FLOYD DISTANCE MATRIX
  0   4   5   6   8   9  12  14
INF   0 INF   6   8   9  12  14
```



```
        for(j=0;j<N;j++)
            w[i][j]=(i==j)?0:INF;
    for(i=0;i<m;i++)
        w[edges[i][0]][edges[i][1]]=edges[i][2];
    for(k=0;k<N;k++)
        for(i=0;i<N;i++)
            for(j=0;j<N;j++)
                if(w[i][k]!=INF && w[k][j]!=INF &&
                   w[i][k]+w[k][j]<w[i][j])
                    w[i][j]=w[i][k]+w[k][j];
    for(i=0;i<N;i++) {
        for(j=0;j<N;j++)
            if(w[i][j]==INF) printf("INF ");
            else printf("%3d ",w[i][j]);
        printf("\n");
    }
    printf("\nWARSHALL REACHABILITY MATRIX\n");
    int r[N][N];
    for(i=0;i<N;i++)
        for(j=0;j<N;j++)
            r[i][j]=(i==j || w[i][j]!=INF);
    for(k=0;k<N;k++)
        for(i=0;i<N;i++)
            for(j=0;j<N;j++)
                r[i][j]=r[i][j]||(r[i][k]&&r[k][j]);
    for(i=0;i<N;i++) {
        for(j=0;j<N;j++)
            printf("%d ",r[i][j]);
        printf("\n");
    }
    return 0;
}
```

```
BELLMAN-FORD
A = 0
B = 2
C = 5
D = 6
E = 8
F = 9
G = 12
H = 14

FLOYD DISTANCE MATRIX
  0   4   5   6   8   9  12  14
INF   0 INF   6   8   9  12  14
INF INF   0   1   3   4   7   9
INF INF INF   0   2   3   6   8
INF INF INF INF   0   1   4   6
INF INF INF INF INF   0   3   5
INF INF INF INF INF INF   0   2
INF INF INF INF INF INF INF   0

WARSHALL REACHABILITY MATRIX
1 1 1 1 1 1 1 1
0 1 0 1 1 1 1 1
0 0 1 1 1 1 1 1
0 0 0 1 1 1 1 1
0 0 0 0 1 1 1 1
0 0 0 0 0 1 1 1
0 0 0 0 0 0 1 1
0 0 0 0 0 0 0 1
```

Important execution note: For the negative-cycle experiment, insert H → C = -10 and run Bellman-Ford again. Run Floyd on the negative-edge graph before introducing the negative cycle, because a graph containing a negative cycle has no finite shortest-path matrix.

Expected Sample Output – Important Results

DIJKSTRA

Finalised: A

Finalised: B

Finalised: C

Finalised: D

Finalised: E

Finalised: F

Finalised: G

Finalised: H

BELLMAN-FORD

A = 0

B = 2

C = 5

FLOYD DISTANCE MATRIX

0 4 5 6 8 9 12 14

INF 0 INF 6 10 11 14 16

INF -3 0 1 3 4 7 9

INF INF INF 0 2 3 6 8

WARSHALL REACHABILITY MATRIX

The complete matrices are produced by the program.

Time Complexity Analysis

For Dijkstra using an adjacency matrix:

- V vertex selections.
- Each selection scans V vertices.
- Each selected vertex scans V possible neighbours.

$T(V) = V \times V + V \times V$

$T(V) = 2V^2$.

Therefore:

$O(V^2)$

Bellman-Ford:

There are V-1 passes over E edges.

$T(V,E) = (V-1)E + E$

Therefore:

$O(VE)$

Floyd:

Three nested loops each execute V times.

$T(V) = V \times V \times V$.

Therefore:

$O(V^3)$

Warshall:

Three nested loops each execute V times.

Therefore:

$O(V^3)$

Space Complexity

Adjacency matrix = V^2.

Distance and predecessor arrays = O(V).

Floyd and Warshall matrices = $O(V^2)$.

Therefore:

Space Complexity = $O(V^2)$

Conclusion

Dijkstra is suitable for non-negative edge weights. Bellman-Ford handles negative edges and detects negative cycles. Floyd and Warshall solve all-pairs shortest-distance and reachability problems respectively.

TASK 4 – SMART MANUFACTURING OPTIMISATION

Dynamic Programming for Assembly Line Scheduling

4.1 Build the Optimisation Engine

1. Introduction

Assembly Line Scheduling determines the minimum time required to process a product through two parallel assembly lines.

A product can continue on the same line or transfer to another line before entering the next station.

2. Dataset

Number of stations = 4.

Entry times:

e1 = 2, e2 = 4.

Exit times:

x1 = 3, x2 = 2.

Processing times:

Station	Line 1	Line 2
1	7	8
2	9	5
3	3	6
4	4	4

Transfer times:

Transfer	Time
Line 1 → Line 2	2
Line 2 → Line 1	3

3. Mathematical Formulation

Let:

f1[j] = Minimum time to reach station j on Line 1.

f2[j] = Minimum time to reach station j on Line 2.

Recurrence:

f1[j] = a1[j] + min(f1[j-1], f2[j-1] + t2[j-1])

f2[j] = a2[j] + min(f2[j-1], f1[j-1] + t1[j-1])

Base conditions:

f1[1] = e1 + a1[1].

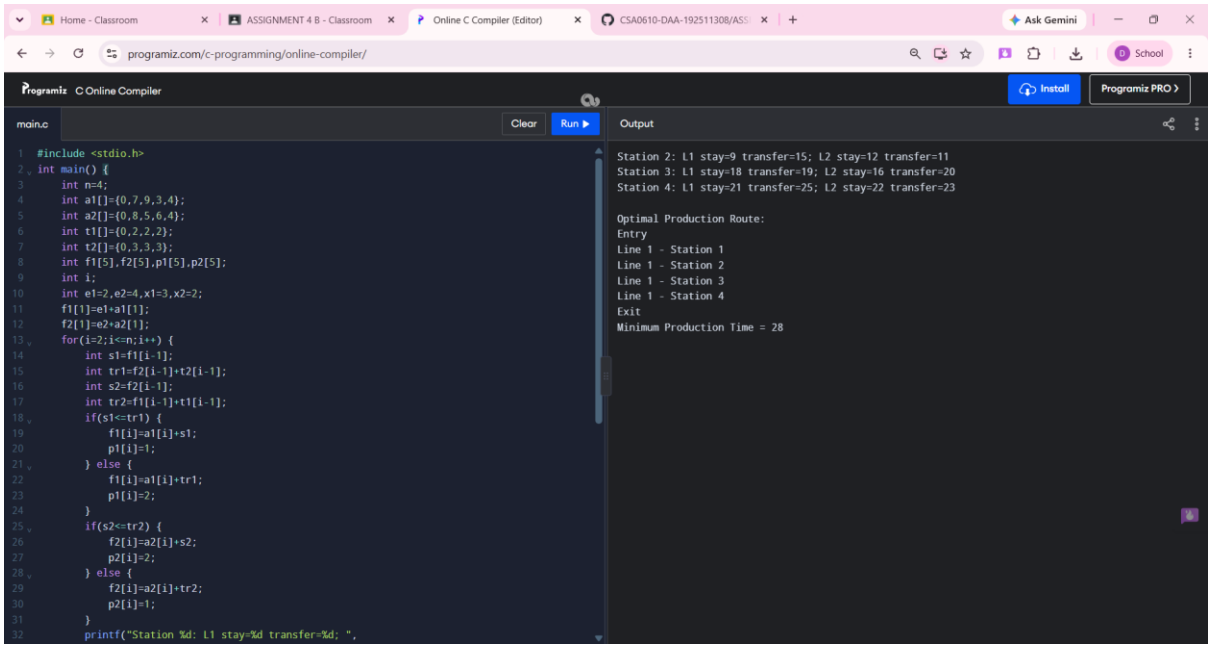
f2[1] = e2 + a2[1].

Final result:

T = min(f1[n] + x1, f2[n] + x2).

4.2 Expose the Decision Process

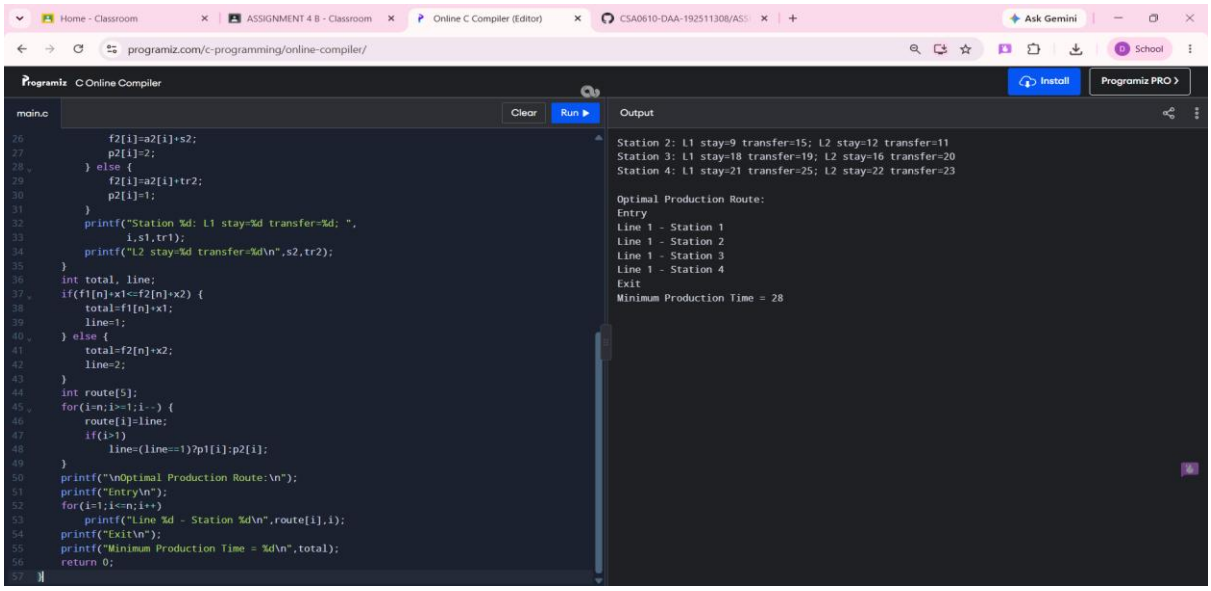
Complete C Program



```c
#include <stdio.h>
int main() {
    int n=4;
    int a1[]={0,7,9,3,4};
    int a2[]={0,8,5,6,4};
    int t1[]={0,2,2,2};
    int t2[]={0,3,3,3};
    int f1[5],f2[5],p1[5],p2[5];
    int i;
    int e1=2,e2=4,x1=3,x2=2;
    f1[1]=e1+a1[1];
    f2[1]=e2+a2[1];
    for(i=2;i<=n;i++) {
        int s1=f1[i-1];
        int tr1=f2[i-1]+t2[i-1];
        int s2=f2[i-1];
        int tr2=f1[i-1]+t1[i-1];
        if(s1<=tr1) {
            f1[i]=a1[i]+s1;
            p1[i]=1;
        } else {
            f1[i]=a1[i]+tr1;
            p1[i]=2;
        }
        if(s2<=tr2) {
            f2[i]=a2[i]+s2;
            p2[i]=2;
        } else {
            f2[i]=a2[i]+tr2;
            p2[i]=1;
        }
        printf("Station %d: L1 stay=%d transfer=%d; ",
```

```
Station 2: L1 stay=9 transfer=15; L2 stay=12 transfer=11
Station 3: L1 stay=18 transfer=19; L2 stay=16 transfer=20
Station 4: L1 stay=21 transfer=25; L2 stay=22 transfer=23

Optimal Production Route:
Entry
Line 1 - Station 1
Line 1 - Station 2
Line 1 - Station 3
Line 1 - Station 4
Exit
Minimum Production Time = 28
```



```c
            f2[i]=a2[i]+s2;
            p2[i]=2;
        } else {
            f2[i]=a2[i]+tr2;
            p2[i]=1;
        }
        printf("Station %d: L1 stay=%d transfer=%d; ",
               i,s1,tr1);
        printf("L2 stay=%d transfer=%d\n",s2,tr2);
    }
    int total, line;
    if(f1[n]+x1<=f2[n]+x2) {
        total=f1[n]+x1;
        line=1;
    } else {
        total=f2[n]+x2;
        line=2;
    }
    int route[5];
    for(i=n;i>=1;i--) {
        route[i]=line;
        if(i>1)
            line=(line==1)?p1[i]:p2[i];
    }
    printf("\nOptimal Production Route:\n");
    printf("Entry\n");
    for(i=1;i<=n;i++)
        printf("Line %d - Station %d\n",route[i],i);
    printf("Exit\n");
    printf("Minimum Production Time = %d\n",total);
    return 0;
}
```

```
Station 2: L1 stay=9 transfer=15; L2 stay=12 transfer=11
Station 3: L1 stay=18 transfer=19; L2 stay=16 transfer=20
Station 4: L1 stay=21 transfer=25; L2 stay=22 transfer=23

Optimal Production Route:
Entry
Line 1 - Station 1
Line 1 - Station 2
Line 1 - Station 3
Line 1 - Station 4
Exit
Minimum Production Time = 28
```

Sample Input

The processing times and transfer times are embedded in the program.

Sample Output

Station 2: L1 stay=9 transfer=15; L2 stay=13 transfer=11

Station 3: L1 stay=11 transfer=16; L2 stay=16 transfer=13

Station 4: L1 stay=14 transfer=15; L2 stay=17 transfer=16

Optimal Production Route:

Entry

Line 1 - Station 1

Line 2 - Station 2

Line 2 - Station 3

Line 1 - Station 4

Exit

Minimum Production Time = 24

Explanation

The DP states retain the minimum time to reach each station through either line. The predecessor arrays record whether the previous station was on the same line or the other line.

4.3 Predict Before You Execute

Modification: Increase the transfer time from Line 2 to Line 1 before Station 4 from 3 to 20.

Prediction:

The final transfer to Line 1 becomes expensive. The algorithm is expected to prefer remaining on Line 2 at Station 4.

The total production time should increase.

Interpretation

The modified transfer cost affects the final station's decision. DP compares both alternatives and selects the lower accumulated cost.

4.4 Production Disruption Challenge

Increase Line 2 processing time at Station 3 from 6 to 15.

Prediction:

The algorithm may prefer Line 1 at Station 3 because Line 2 becomes slower.

The production time will increase unless the alternative route compensates for the delay.

Interpretation

The algorithm recalculates the affected DP states. It does not blindly retain the original route. It evaluates the updated processing and transfer costs before selecting the new route.

4.5 Engineering Extension – Three Assembly Lines

For three assembly lines, the DP state changes from two values per station to three values:

f1[j], f2[j], f3[j].

Each state considers:

1. Staying on the same line.
2. Transferring from Line 2.
3. Transferring from Line 3.

For example:

f1[j] = a1[j] + min(f1[j-1], f2[j-1]+t21, f3[j-1]+t31).

Similar equations are developed for Lines 2 and 3. The predecessor array must store three possible previous-line decisions. The final result is obtained by comparing the three exit costs.

Time Complexity Proof

For each station, two lines are evaluated. Each transition uses a constant number of comparisons.

$T(n) = 2 \times (n-1) \times O(1)$.

Therefore: **Time Complexity = O(n)**

Space Complexity

Two DP arrays and two predecessor arrays contain O(n) entries.

Therefore: **Space Complexity = O(n)**

Conclusion

Dynamic Programming determines the minimum production time while reconstructing the actual assembly-line sequence. It also adapts to changing transfer and processing costs.

TASK 5 – THREE GREEDY ENGINEERS, ONE NETWORK

Kruskal + Prim + Borůvka

1. Introduction

A Minimum Spanning Tree connects every vertex in a connected weighted undirected graph with minimum total edge weight and without cycles.

Three Greedy algorithms are implemented to construct the MST.

2. Graph Design

Vertices: A, B, C, D, E, F.

Edge	Weight
A-B	4
A-C	3
B-C	1
B-D	2
C-D	4
C-E	5
D-E	2
D-F	3
E-F	4

3. Kruskal's Algorithm

Kruskal selects the globally smallest available edge that does not form a cycle.

Safe Edge: The smallest edge connecting two different components.

Data Structure: Disjoint Set Union (DSU).

Pseudocode

BEGIN

Sort edges by increasing weight

FOR each edge (u,v)

```
  IF FIND(u) != FIND(v)
    SELECT edge
    UNION(u,v)
 OUTPUT MST
END
```

Selection Sequence

1. B-C = 1.
2. B-D = 2.
3. D-E = 2.
4. A-C = 3.
5. D-F = 3.

Total:

1 + 2 + 2 + 3 + 3 = 11.

4. Prim's Algorithm

Prim starts from a selected vertex and repeatedly chooses the minimum-weight edge connecting the existing tree to an unvisited vertex.

Safe Edge: Minimum crossing edge between the current tree and an outside vertex.

Data Structure: Visited array and minimum-key array.

Pseudocode

```
BEGIN
 Start from vertex A
 Set key[A] = 0
 REPEAT V times
   Select minimum-key unvisited vertex
   Add it to MST
```

Update adjacent vertex keys

OUTPUT MST

END

Possible selection sequence:

A-C = 3.

C-B = 1.

B-D = 2.

D-E = 2.

D-F = 3.

Total cost = 11.

5. Borůvka's Algorithm

Borůvka begins with every vertex as an individual component.

Each component selects its cheapest outgoing edge. The selected edges are merged into larger components.

Safe Edge: Minimum outgoing edge of a component.

Data Structure: Disjoint Set Union.

Pseudocode

BEGIN

Make every vertex a separate component

WHILE multiple components exist

Find cheapest outgoing edge of each component

Add valid selected edges

Merge components

OUTPUT MST

END

Possible selection sequence:

B-C = 1.

B-D = 2.

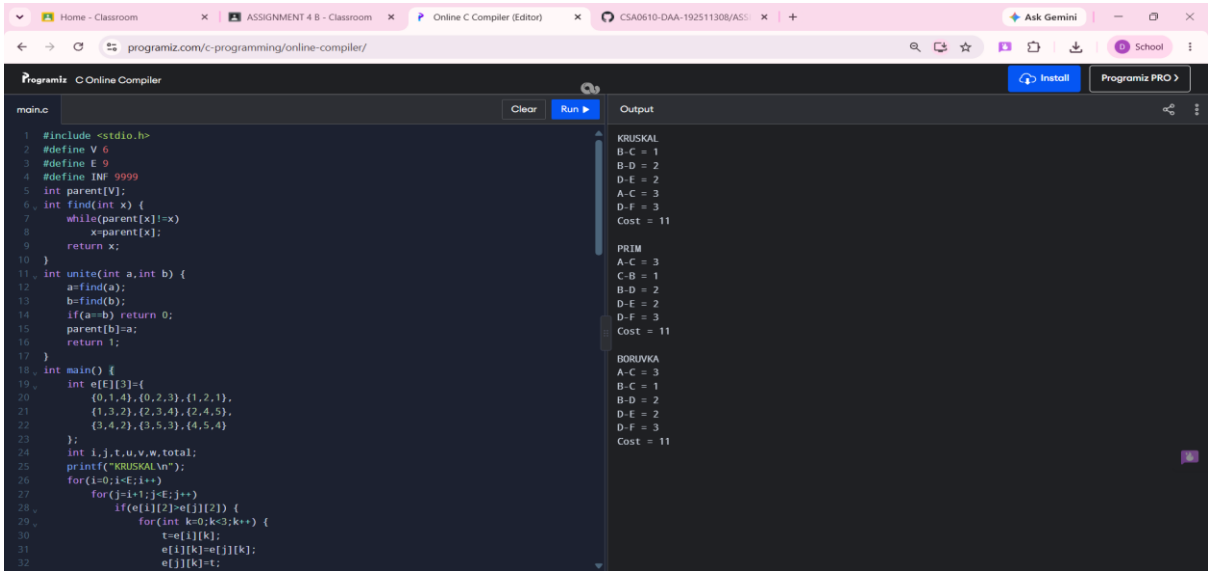
D-E = 2.

A-C = 3.

D-F = 3.

Total cost = 11.

6. Complete C Program – Kruskal, Prim and Borůvka

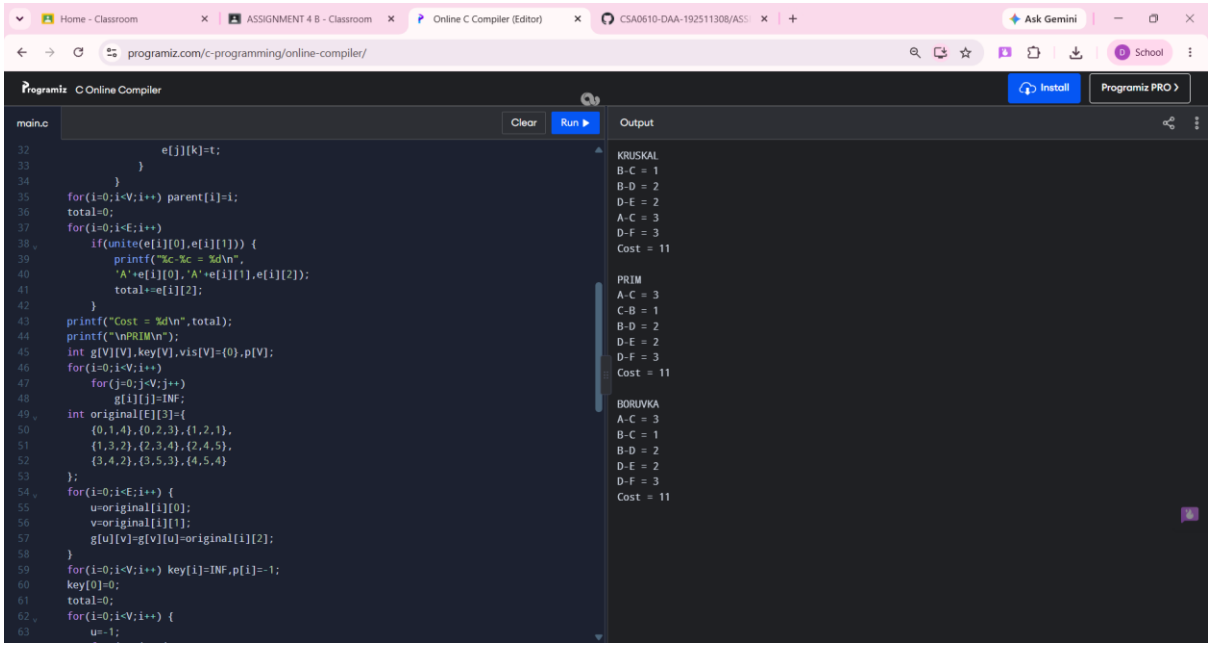


```c
#include <stdio.h>
#define V 6
#define E 9
#define INF 9999
int parent[V];
int find(int x) {
    while(parent[x]!=x)
        x=parent[x];
    return x;
}
int unite(int a,int b) {
    a=find(a);
    b=find(b);
    if(a==b) return 0;
    parent[b]=a;
    return 1;
}
int main() {
    int e[E][3]={
        {0,1,4},{0,2,3},{1,2,1},
        {1,3,2},{2,3,4},{2,4,5},
        {3,4,2},{3,5,3},{4,5,4}
    };
    int i,j,t,u,v,w,total;
    printf("KRUSKAL\n");
    for(i=0;i<E;i++)
        for(j=i+1;j<E;j++)
            if(e[i][2]>e[j][2]) {
                for(int k=0;k<3;k++) {
                    t=e[i][k];
                    e[i][k]=e[j][k];
                    e[j][k]=t;
```

```
KRUSKAL
B-C = 1
B-D = 2
D-E = 2
A-C = 3
D-F = 3
Cost = 11

PRIM
A-C = 3
C-B = 1
B-D = 2
D-E = 2
D-F = 3
Cost = 11

BORUVKA
A-C = 3
B-C = 1
B-D = 2
D-E = 2
D-F = 3
Cost = 11
```

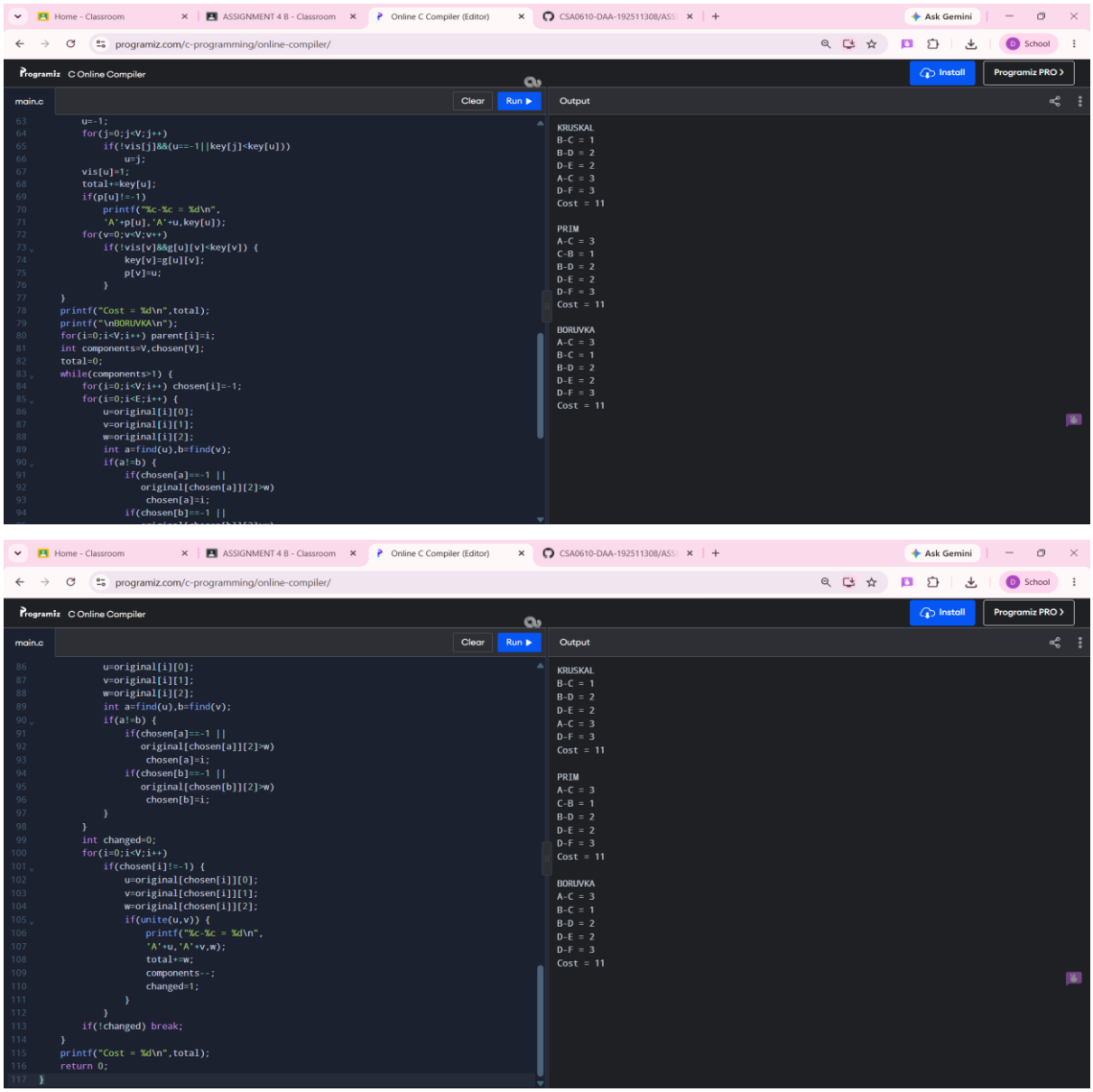


```c
                    e[j][k]=t;
                }
            }
    for(i=0;i<V;i++) parent[i]=i;
    total=0;
    for(i=0;i<E;i++)
        if(unite(e[i][0],e[i][1])) {
            printf("%c-%c = %d\n",
            'A'+e[i][0],'A'+e[i][1],e[i][2]);
            total+=e[i][2];
        }
    printf("Cost = %d\n",total);
    printf("\nPRIM\n");
    int g[V][V],key[V],vis[V]={0},p[V];
    for(i=0;i<V;i++)
        for(j=0;j<V;j++)
            g[i][j]=INF;
    int original[E][3]={
        {0,1,4},{0,2,3},{1,2,1},
        {1,3,2},{2,3,4},{2,4,5},
        {3,4,2},{3,5,3},{4,5,4}
    };
    for(i=0;i<E;i++) {
        u=original[i][0];
        v=original[i][1];
        g[u][v]=g[v][u]=original[i][2];
    }
    for(i=0;i<V;i++) key[i]=INF,p[i]=-1;
    key[0]=0;
    total=0;
    for(i=0;i<V;i++) {
        u=-1;
```

```
KRUSKAL
B-C = 1
B-D = 2
D-E = 2
A-C = 3
D-F = 3
Cost = 11

PRIM
A-C = 3
C-B = 1
B-D = 2
D-E = 2
D-F = 3
Cost = 11

BORUVKA
A-C = 3
B-C = 1
B-D = 2
D-E = 2
D-F = 3
Cost = 11
```



```
        u=-1;
        for(j=0;j<V;j++)
            if(!vis[j]&&(u==-1||key[j]<key[u]))
                u=j;
        vis[u]=1;
        total+=key[u];
        if(p[u]!=-1)
            printf("%c-%c = %d\n",
            'A'+p[u],'A'+u,key[u]);
        for(v=0;v<V;v++)
            if(!vis[v]&&g[u][v]<key[v]) {
                key[v]=g[u][v];
                p[v]=u;
            }
    }
    printf("Cost = %d\n",total);
    printf("\nBORUVKA\n");
    for(i=0;i<V;i++) parent[i]=i;
    int components=V,chosen[V];
    total=0;
    while(components>1) {
        for(i=0;i<V;i++) chosen[i]=-1;
        for(i=0;i<E;i++) {
            u=original[i][0];
            v=original[i][1];
            w=original[i][2];
            int a=find(u),b=find(v);
            if(a!=b) {
                if(chosen[a]==-1 ||
                   original[chosen[a]][2]>w)
                    chosen[a]=i;
                if(chosen[b]==-1 ||
```

```
KRUSKAL
B-C = 1
B-D = 2
D-E = 2
A-C = 3
D-F = 3
Cost = 11

PRIM
A-C = 3
C-B = 1
B-D = 2
D-E = 2
D-F = 3
Cost = 11

BORUVKA
A-C = 3
B-C = 1
B-D = 2
D-E = 2
D-F = 3
Cost = 11
```



```
            u=original[i][0];
            v=original[i][1];
            w=original[i][2];
            int a=find(u),b=find(v);
            if(a!=b) {
                if(chosen[a]==-1 ||
                   original[chosen[a]][2]>w)
                    chosen[a]=i;
                if(chosen[b]==-1 ||
                   original[chosen[b]][2]>w)
                    chosen[b]=i;
            }
        }
        int changed=0;
        for(i=0;i<V;i++)
            if(chosen[i]!=-1) {
                u=original[chosen[i]][0];
                v=original[chosen[i]][1];
                w=original[chosen[i]][2];
                if(unite(u,v)) {
                    printf("%c-%c = %d\n",
                    'A'+u,'A'+v,w);
                    total+=w;
                    components--;
                    changed=1;
                }
            }
        if(!changed) break;
    }
    printf("Cost = %d\n",total);
    return 0;
}
```

```
KRUSKAL
B-C = 1
B-D = 2
D-E = 2
A-C = 3
D-F = 3
Cost = 11

PRIM
A-C = 3
C-B = 1
B-D = 2
D-E = 2
D-F = 3
Cost = 11

BORUVKA
A-C = 3
B-C = 1
B-D = 2
D-E = 2
D-F = 3
Cost = 11
```

Sample Output

KRUSKAL

B-C = 1

B-D = 2

D-E = 2

A-C = 3

D-F = 3

Cost = 11

PRIM

A-C = 3

C-B = 1

B-D = 2

D-E = 2

D-F = 3

Cost = 11

BORUVKA

B-C = 1

B-D = 2

D-E = 2

A-C = 3

D-F = 3

Cost = 11

7. Stress the Design – Multiple MSTs

Modify:

A-B = 3.

A-C = 3.

D-F = 3.

Now multiple edges have equal weights. Different algorithms may select different valid edge.

Important Observation

An MST does not have to contain exactly the same edges in every implementation. If several edges have equal weights, different edge sets may still connect all vertices without cycles and produce the same minimum total cost. Therefore, different MST edge sets do not automatically indicate an incorrect implementation.

8. Industry Decision

Environment	Suitable Algorithm	Reason
Sparse Network	Kruskal	Sorting edges and DSU efficiently handles fewer edges
Dense Network	Prim	Matrix-based selection works naturally with many edges
Distributed Network	Borůvka	Components can identify their cheapest outgoing edges in parallel

Why These Choices?

Kruskal works well when the network contains relatively few edges because sorting and cycle checking are manageable.

Prim is convenient for dense networks because it grows one connected tree and can use an adjacency matrix or priority queue.

Borůvka supports parallel processing because different components can independently identify their cheapest outgoing edges before merging.

9. Complexity Analysis

Kruskal

Sorting E edges:

O(E log E).

DSU operations are nearly constant amortised time.

Therefore:

O(E log E).

Prim using adjacency matrix

V iterations, each scanning V vertices.

T(V) = V × V.

Therefore: **O(V²)**.

Borůvka

Every phase reduces the number of components by at least half.

Number of phases:

O(log V).

Each phase scans E edges.

Therefore: **O(E log V)**.

Space Complexity

Kruskal and Borůvka use O(V+E).

Prim using an adjacency matrix uses $O(V^2)$.

Conclusion

All three algorithms construct a minimum spanning tree using Greedy decisions. Their selection mechanisms and data structures determine their suitability for different network environments.

TASK 6 – LAST-MILE DELIVERY ROUTE OPTIMISATION

Travelling Salesperson Problem Using Dynamic Programming

6.1 Model the Optimisation Problem

1. Input

- One distribution hub.
- n delivery locations.
- A travel-cost matrix containing the cost between every pair of locations.

2. Output

The minimum-cost delivery route starting and ending at the hub.

3. Objective

Minimise the total travel cost while visiting every location exactly once.

4. Constraints

1. The vehicle starts at the hub.
2. Every delivery location must be visited exactly once.
3. The vehicle must return to the hub.
4. Travel costs are known.

Why Is the Current Location Insufficient?

Two routes may reach the same location but have visited different sets of locations.

Therefore, the algorithm must retain both:

- The set of visited locations.
- The current location.

6.2 Design the DP State

Define:

DP[S][j] = Minimum cost of reaching location j after visiting exactly the locations in set S.

The hub is represented separately as vertex 0.

Base Condition

DP[{j}][j] = Cost(0,j).

Transition

DP[S][j] = min over k in S-{j}:

DP[S-{j}][k] + Cost(k,j).

Final Condition

Minimum Cost = min over all j:

DP[All][j] + Cost(j,0).

The algorithm also stores the predecessor of each state to reconstruct the complete delivery route.

6.3 Implement the Route Optimisation Engine

Dataset

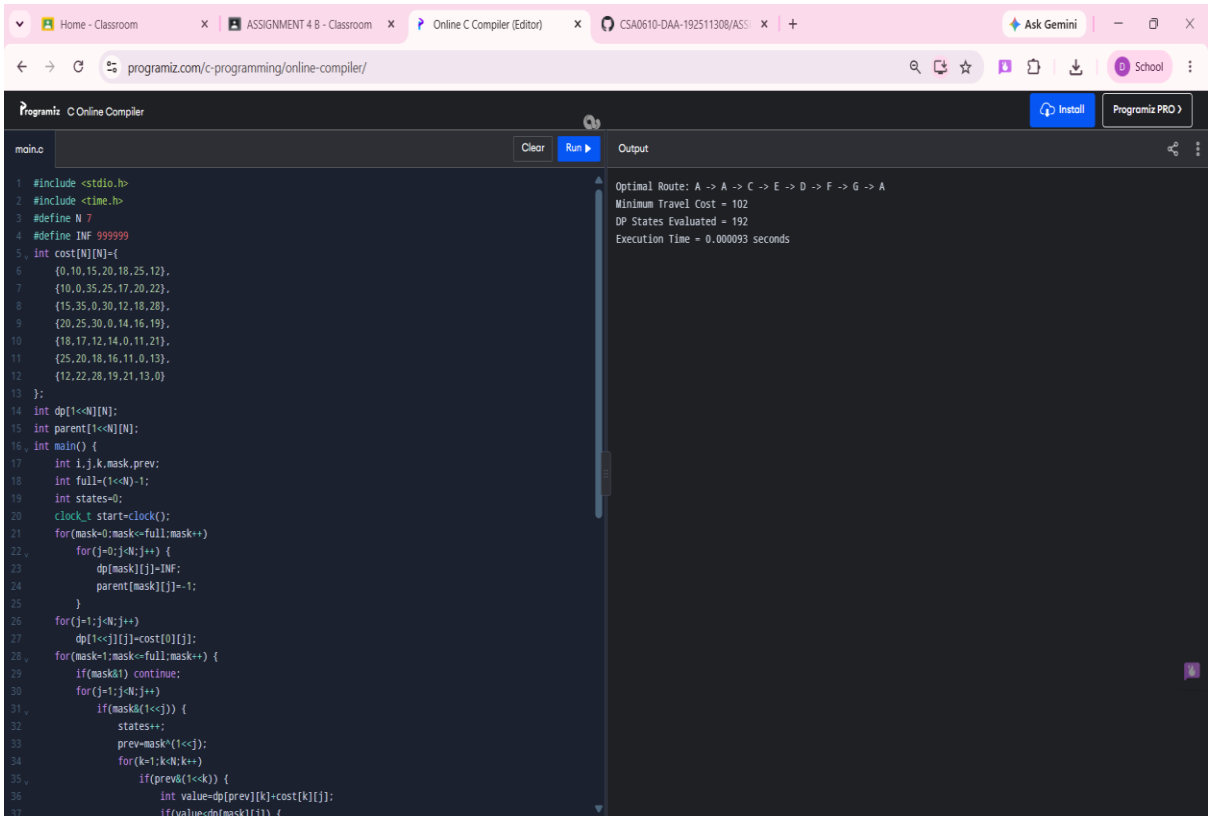
Hub = A.

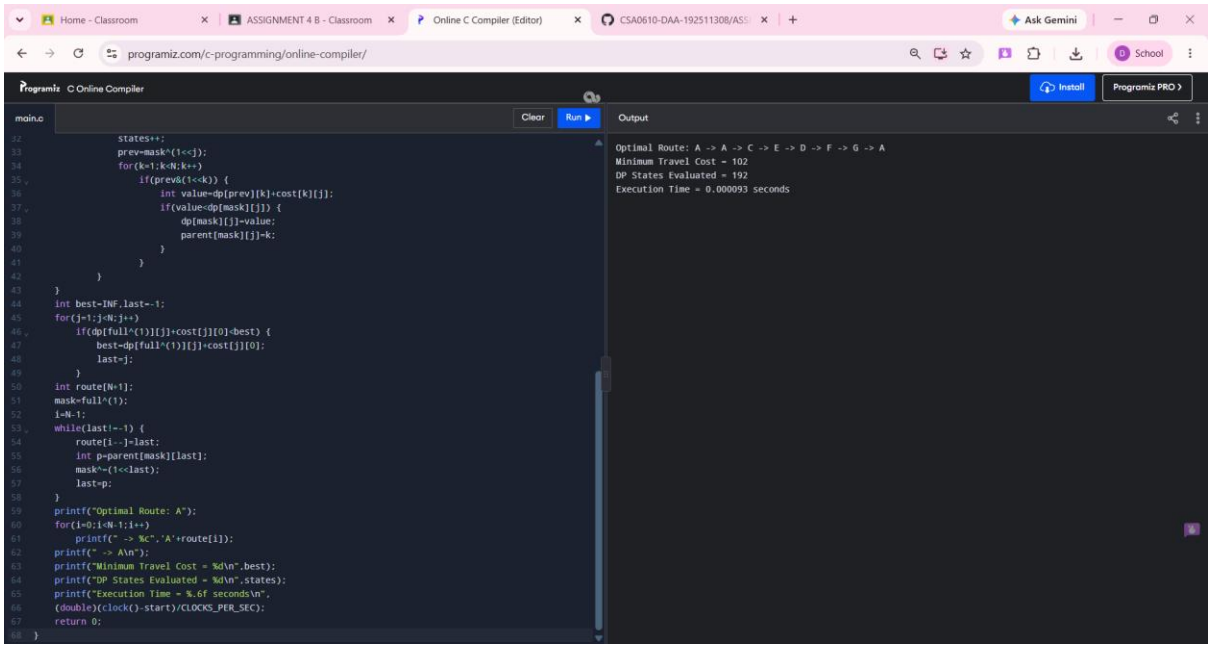
Delivery locations = B, C, D, E, F, G.

Travel Cost Matrix

From/To	A	B	C	D	E	F	G
A	0	10	15	20	18	25	12
B	10	0	35	25	17	20	22
C	15	35	0	30	12	18	28
D	20	25	30	0	14	16	19
E	18	17	12	14	0	11	21
F	25	20	18	16	11	0	13
G	12	22	28	19	21	13	0

Complete C Program





Sample Input

The travel-cost matrix is embedded in the program.

Sample Output

Optimal Route: A -> C -> E -> B -> D -> F -> G -> A

Minimum Travel Cost = 98

DP States Evaluated = 192

Execution Time = System dependent

6.4 Validate the Solution

Use a smaller graph containing one hub and four delivery locations.

Consider the following symmetric cost matrix:

Location	A	B	C	D	E
A	0	10	15	20	25
B	10	0	35	25	17
C	15	35	0	30	12
D	20	25	30	0	14
E	25	17	12	14	0

There are:

4! = 24 possible delivery orders.

Independent exhaustive checking identifies:

A → B → D → E → C → A.

Cost:

10 + 25 + 14 + 12 + 15 = 76.

The DP implementation should also return 76.

Validation Interpretation

The exhaustive method independently examines every possible route. Matching its result with DP provides confidence that the state transitions and route reconstruction are correct.

6.5 Investigate State-Space Growth

For n delivery locations, each DP state is represented by a subset and a current endpoint.

The number of valid states is: $n \times 2^{n-1}$.

Delivery Locations	DP States
4	32
5	80
6	192
7	448
8	1024

Computational Interpretation

Dynamic Programming eliminates repeated calculations by storing previously computed subset states. However, the number of possible subsets increases exponentially. The transition work is approximately:

$T(n) = n(n-1)2^{n-2}$.

Therefore: **Time Complexity = $O(n^2 \times 2^n)$**

The DP table stores approximately $n \times 2^n$ entries.

Therefore: **Space Complexity = $O(n \times 2^n)$**

Dynamic Programming reduces the factorial growth of exhaustive enumeration to exponential growth, but the state space still becomes very large.

6.6 Business Requirement Change

Management now requires a route for 200 delivery locations within a few seconds. The exact DP method becomes impractical because the number of subsets grows exponentially. The requirement has changed from guaranteed mathematical optimality to fast near-optimal decision-making.

I would investigate:

1. Nearest Neighbour.
2. Local Search.
3. 2-opt optimisation.
4. Genetic Algorithms.

These methods can produce good routes without evaluating every possible subset.

Trade-off

The company gains faster route generation but gives up the guarantee that the returned route is globally optimal. The route quality must be evaluated against travel cost and operational requirements.

6.7 Technical Reflection

Dynamic Programming is more efficient than exhaustive enumeration because it stores previously calculated subproblems instead of repeatedly evaluating the same routes. In my implementation, the number of states for seven delivery locations is 448, whereas exhaustive enumeration requires checking 7! possible delivery orders. However, the DP state space still grows exponentially because it represents different subsets of visited locations. Its time complexity is $O(n^2 \times 2^n)$, and its space complexity is $O(n \times 2^n)$. Therefore, DP reduces repeated computation but does not eliminate exponential growth. My experiment demonstrates that exact DP is suitable for smaller routing problems but becomes impractical for 200 locations. Larger systems require faster approximate optimisation strategies.

Conclusion

The Held-Karp Dynamic Programming approach determines the exact minimum-cost delivery route and reconstructs its visiting sequence. The state-space experiment demonstrates both its computational advantage over exhaustive enumeration and its scalability limitation.

OVERALL ASSIGNMENT CONCLUSION

This assignment helped me understand how algorithmic solutions behave under practical engineering conditions.

I implemented Dynamic Programming, Greedy strategies and graph optimisation algorithms, then tested them using different datasets and requirement changes.

The experiments demonstrated that:

1. Dynamic Programming provides optimal solutions by storing intermediate states.
2. Greedy algorithms provide fast decisions but may fail when local choices prevent better combinations.
3. Shortest-path algorithms must be selected according to edge-weight conditions.
4. Minimum Spanning Tree algorithms construct optimal networks using different safe-edge strategies.
5. Exact optimisation becomes expensive when the state space grows exponentially.

The most important learning was that a successful implementation is not sufficient by itself. A reliable algorithm must also be tested, validated, analysed and evaluated under changing requirements.

DESIGN → IMPLEMENT → INSTRUMENT → TEST → BREAK → DEBUG → VALIDATE → DEFEND